SwiftAid

Aid that arrives before it spoils.

Project summary: what was built, how every panel works, the algorithms behind it, and where it can go next.

At a glance	
Problem	The Last-Mile Spoilage Trap: coordinating perishable surplus and fragile aid under uncertainty
Solution	A real-time dispatch and safety-enforcement platform joining donors, volunteer riders and shelters
Demo city	Pune (fictional organisations at real neighbourhoods): 5 donors, 6 shelters, 10 riders
Surfaces	Control room, donor, rider, shelter, impact analytics, live board, judge mode, guided demo
Core ideas	Leases against ghost reservations · live re-planning · rule-checked, quantum-inspired optimizers · AI proposes, engine disposes
Quality	100 unit/property tests (incl. seeded fuzzing) · 48 browser checks · lint, types and production build clean
Repository	github.com/veeerrh/SwiftAid_abraxas

How to use this document

Part 1 explains the idea. Parts 2-4 cover architecture, frameworks and algorithms. Part 5 walks through every screen and panel with a **Change ideas** line for each. Part 6 lists the API and data model. Part 8 collects further ideas, grouped by area, to help you decide what to change next.

1 · The problem and our answer

Every night restaurants, hostel messes and pharmacies try to donate food and supplies, and volunteers on two-wheelers carry it to shelters. Nearly 40% is wasted or harmful because of four coordination blind spots. SwiftAid treats **time, transport capacity, compatibility and human unreliability as live variables**.

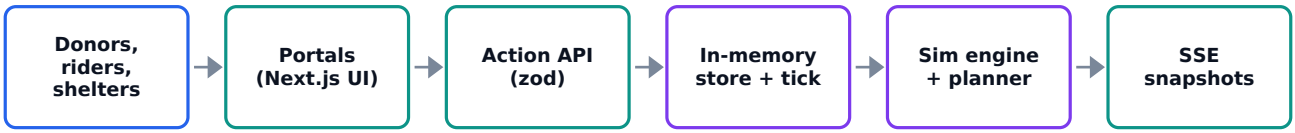
Blind spot in the brief	What goes wrong	How SwiftAid handles it
Supply, rider and shelter mismatch	180 meals, but a scooter carries 40 and Shelter A fits 50	The planner **splits** every batch across riders and shelters (one trip per rider), bounded by rider capacity, remaining shelter capacity and the batch's expiry
Ghost reservations	A claim stalls; by the time it is cancelled the food is nearly spoiled	Every claim is a **lease** whose length shrinks with remaining shelf life. Missed heartbeats release it; a **hot standby** or the planner reassigns
Conditions change mid-journey	A citizen fills the shelter while the rider is halfway there	An event-driven **re-planner** diverts in-transit riders to the best feasible shelter, or flags them stranded with the reason
Dependency rules	Formula to a site with no clean water; raw rations with no fuel	A hard-rule layer plus a **safety validator** that every plan must pass, whoever produced it

Scoring lens used throughout

Criterion	Weight	Where we invest
Functionality and execution	30%	Deterministic engine, conservation invariants, fuzz tests, offline fallbacks
UI and UX polish	20%	Token-driven design system, live map, countdown rings, role portals, mobile layouts
Solution feasibility	20%	Impact page, scale path, honest limitations, documented architecture
Innovation and creativity	15%	QIGA/QPSO inside the rule engine, Monte Carlo survival, multimodal intake, simulate-then-approve copilot
Live demonstration	15%	Guided demo, chaos panel, judge mode, live board

2 · Architecture

One Next.js application. The simulation engine is plain TypeScript with no framework dependencies, so it is fast to test and easy to move.



Layer	Location	Responsibility
Domain	src/lib/domain	Types, geo maths, compatibility rules, seeded RNG, Pune seed data
Engine	src/lib/engine	Feasibility, greedy baseline, QIGA/QPSO, Monte Carlo, validator, simulation (leases, diverts, chaos)
Store	src/lib/store.ts	Process-wide singleton; 500 ms ticker converts wall time to sim time; broadcasts snapshots
Actions	src/lib/actions.ts	The only way state changes: ~25 zod-validated action types shared by UI, copilot and tests
AI helpers	src/lib/intake.ts, copilot.ts	Donation parsing, copilot proposals and plan-diff simulation on a cloned city
API routes	src/app/api/*	state, stream (SSE), action, intake, copilot
UI	src/app, src/components	Portals, panels, design-system primitives, live map

Time model

All times are **simulation minutes**. By default 1 real second = 0.25 simulated minutes, so a 75-minute shelf life lasts five real minutes. The control room offers 1×, 2×, 4× and 8× (0.25, 0.5, 1, 2 sim-min per second). The engine steps in slices of at most 0.25 sim-minutes, so behaviour is identical at any speed and in tests.

Real-time delivery

Browsers open one server-sent-events stream. Each tick (and each action) pushes a full state snapshot (about 30 KB). Client markers are extrapolated between snapshots for smooth movement. If SSE fails, the client polls `/api/state` every 1.5 s.

> **Design rule: AI proposes, the engine disposes**
>
> Nothing from a language model can change the city directly. Intake output is sanitised, reviewed by the donor and re-validated; copilot actions are restricted to an allow-list, simulated on a copy of the city, shown as a diff, and applied only on human approval through the same validated path as a button click.

3 · Frameworks and libraries

Chosen to maximise frontend quality inside a five-hour solo build, with no external services required to run.

Technology	Role in SwiftAid
Next.js 15 (App Router) + React 19	Routing, server routes (API, SSE), server singleton for the live store
TypeScript	End-to-end types; the engine shares types with the UI
Tailwind CSS v4	Utility styling bound to CSS-variable design tokens (dark and light themes)
Framer Motion	Panel entrances, list reordering, animated numbers, drawers, toasts
MapLibre GL	Live map: markers, glowing route lines, focus modes. Basemap from CARTO; falls back to a plain canvas offline
OSRM public router	Road-following route geometry (cached in localStorage); falls back to straight lines
Recharts	Convergence chart in the Optimizer Lab and all analytics charts
Zustand	Client live-state store fed by SSE; toast queue
Zod	Validates every action, intake draft and copilot proposal
Anthropic SDK (optional)	Photo/text intake and copilot reasoning via tool-use. Fully optional: deterministic fallbacks cover every path
Vitest	100 unit, property, simulation and fuzz tests
Playwright (core)	48 browser checks driving the real UI, including mobile-overflow checks
lucide-react, clsx, tailwind-merge	Icons and class composition

Design system

All colours, radii, blur, shadows and motion timings are CSS variables in `globals.css` (dark default, light via `data-theme`). Semantic colours carry meaning everywhere: **teal = safe**, **amber = at risk**, **rose = critical**, **violet = AI / optimizer**, **blue = in flight**. Restyling to a new reference means editing tokens, not components.

4 · Algorithms and their roles

The planner is a pure function over a snapshot of the city. Hard rules filter what is allowed; optimizers search among allowed plans; a validator has the final word.



Algorithm	Role	Key details
Feasibility rules	Decide whether a (cargo, rider, shelter) triple is allowed at all	Carrier ownership; shelter open and accepts category; formula needs water, raw rations need fuel, cold-chain medicine needs cold storage; shelter has space; **arrival ≤ expiry − 5 min** (trip = travel + 2 min pickup + 2 min drop handling)
Travel model	Estimate trip times	Haversine distance × 1.35 road factor ÷ rider speed (scooter 24, bike 21, van 22 km/h), × live traffic factor, minimum 1.5 min per leg
Analytic survival	Cheap success probability used inside optimizer loops	P(arrive before expiry) × rider reliability, with a normal-approximation of delay (σ = 18% of trip time, wider for less reliable riders)
Greedy allocator	Baseline plan and optimizer seed	Most urgent cargo first; repeatedly pick the rider/shelter pair maximising quantity × survival − 0.2 × minutes; one trip per rider
QIGA (quantum-inspired GA)	Search the **discrete** structure: which rider takes which cargo to which shelter	Q-bit angles θ (P(1)=sin²θ); observe → decode → repair; rotation gates move angles toward the best solution; catastrophe step after 8 stagnant generations. Population 28, up to 45 generations, 160 ms budget
QPSO (quantum-behaved swarm)	Tune **continuous** parameters: load fractions and rider priorities (and, in full mode, assignments)	No velocities; particles sample a delta-potential well around φ·pbest+(1−φ)·gbest with width β|mbest−x|ln(1/u); β decays 1.0 → 0.5. Population 26, up to 55 iterations
Hybrid	Best of both	QIGA finds the topology, then QPSO (refine mode, 40 iterations) tunes loads and priorities on it
Fitness function	What the optimizers maximise	10 × expected safe units + placed units − 0.05 × total travel minutes
Genome decoder + repair	Guarantee every candidate is rule-feasible	Per rider: cargo, shelter, load, priority. Infeasible genes fall back to the fastest feasible shelter, so searches only see valid plans
Greedy floor	Never worse than baseline	Optimizer results below the greedy fitness are replaced by the greedy plan; greedy seeds every run (warm start)
Monte Carlo survival	Show the real risk to humans	1,500 samples per leg: log-normal traffic noise, rider ghosting with probability 1−reliability, hot-standby recovery delay. Gives p10/p50/p90 ETAs and a histogram
Hot standby	Pre-empt ghosting on risky legs	When a leg's analytic survival < 85%, the fastest feasible idle rider is reserved; it gets first refusal if the lease is released
Lease and heartbeat	Kill ghost reservations	Lease TTL = clamp(15% of remaining life, 2, 8) min; offer TTL = clamp(10%, 1.5, 4) min; warning after 1.5 min of silence; silent riders stop moving

Algorithm	Role	Key details
Re-planner	React to any change	Find legs whose destination became invalid → release pickup-phase legs, divert drop-phase legs (only their own rider may carry that cargo) → solve → validate → apply
Safety validator	Final gatekeeper	Re-checks rules, one trip per rider, rider capacity, shelter capacity, cargo totals, unknown references. A rejected plan falls back to greedy and is logged
Offline intake parser	Works with no AI key	Keyword categories (dependencies win over food words), quantity and time extraction, risk flags, confidence score
Copilot diff simulation	Preview before acting	Clones the store, runs proposed actions through the real dispatcher, diffs legs and totals; the live city is never touched by a preview

Honest note on 'quantum'

QIGA and QPSO run on ordinary computers. They borrow ideas from quantum mechanics (superposition-like probabilistic bits, potential-well sampling) to explore search spaces. In the demo scenarios they raise expected safe delivery by a few percent over greedy (banquet: 162 → 168.8 expected safe meals, survival 90% → 94%); they matter most when capacity is tight and reliability varies.

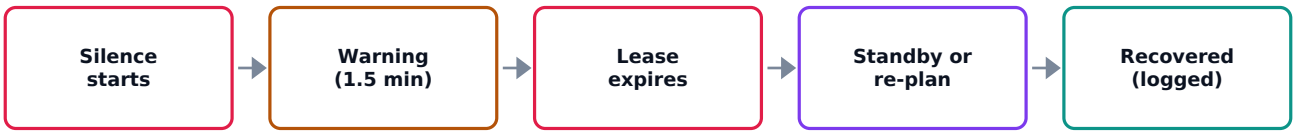
How the engine behaves, step by step

A donation, from posting to delivery



- **Batch posted:** donor, category, quantity, safe window and cold-chain flag. Unplaced quantity starts at the full amount.
- **Plan and split:** cargo is the batch remainder at the donor. The solver produces assignments (rider, shelter, quantity); legs are created in the offered state, and hot standbys and survival estimates are attached.
- **Offer:** simulated riders accept after 0.4 min; human-controlled riders accept or decline in the rider portal. An unanswered offer times out and is reassigned.
- **Move and deliver:** riders interpolate along the route; on arrival the shelter's capacity is reduced and the batch's delivered count increases. Late arrivals are impossible by construction, but cargo that outlives its window while in transit is recorded as spoiled.

A rider ghosts



- Silent riders stop moving (their phase timer is frozen). If cargo was not yet picked up it returns to the donor; if it was, a recovery point is stored at the rider's last position and the replacement rider collects it there.
- The metric **Avg response** is the time from the rider's last heartbeat to the new leg being created.

A shelter closes or fills mid-route



- Pickup-phase legs are simply released and re-planned. Drop-phase legs become divert cargo that only the rider carrying it may take, and only in full (no partial diverts).
- If no safe shelter is reachable the rider is flagged **stranded** with a plain-English reason (for example 'no clean water at Hadapsar; 3 shelters closed'); the planner retries every 2 sim-minutes.

Conservation invariant

Every unit of every batch is always exactly one of: **delivered, spoiled, waiting at the donor/recovery point, or on a rider**. The test suite asserts this after every simulation tick, which is how a real double-booking bug was caught and fixed.

5 · Screens and panels

Every screen shares the same live city. Each card below lists what a panel shows, how it works, where its data and code live, and ideas for changing it.

5.1 Control room /admin

Layout
Full-bleed live map in the background. Left rail: batches and live feed. Centre: KPI strip (top) and dispatch timeline (bottom). Right rail: riders/shelters and the chaos panel. On phones the map sits on top and panels stack below.

Live map	
SHOWS	Donors (bag icons), shelters (ring = free capacity), riders (initial inside a ring), glowing routes coloured by survival, dashed next-leg routes, a rose flash when a rider is diverted.
HOW IT WORKS	Markers sync to each snapshot; positions and routes redraw ~15 fps using a smoothed sim clock and road polylines (OSRM, cached). Rider rings change with status: blue busy, rose ghosting, violet manual, dashed violet standby. Click a donor or rider to focus its batch.
DATA / CODE	components/map/LiveMap.tsx · lib/client/routing.ts · MapLibre + CARTO tiles (offline fallback: plain canvas)
CHANGE IDEAS	Cluster markers when zoomed out; show heat of waiting batches; add a route-ETA tooltip; draw shelter catchment areas.

Top bar	
SHOWS	Brand, navigation between portals, scenario name, sim clock (19:00 start), connection state, theme toggle, Demo / Copilot / Optimizer Lab buttons.
HOW IT WORKS	Clock extrapolates between snapshots; connection state reflects SSE, polling fallback or offline.
DATA / CODE	components/shell/TopBar.tsx · lib/client/live.ts

KPI strip	
SHOWS	Delivered, safe-rate %, spoiled, active legs, ghost recoveries, average response (min), diverts.
HOW IT WORKS	Animated numbers computed from metrics and live legs. Safe rate = delivered ÷ (delivered + spoiled).
DATA / CODE	components/admin/KpiStrip.tsx · state.metrics
CHANGE IDEAS	Add units-per-hour, cost per safe unit, or per-category breakdowns.

Batches rail	
SHOWS	One card per batch: countdown ring (colour by life left), status badge, stacked bar (delivered / moving / waiting / spoiled), Monte Carlo survival chip. Selecting a card expands each leg.
HOW IT WORKS	Expanded legs show rider → shelter, quantity, ETA, a survival histogram against the deadline (green before, rose after), p50/p90 ETA, standby and divert badges. Selecting also dims other routes on the map and timeline.
DATA / CODE	components/admin/BatchRail.tsx · SurvivalBand.tsx · lib/client/derive.ts
CHANGE IDEAS	Sort/filter by urgency or donor; show cargo category icons; add batch-level actions (cancel, expedite).

Riders and Shelters rail

SHOWS	Riders: status, vehicle, capacity, reliability, heartbeat age, current delivery, standby / human-controlled badges, Go silent / Revive. Shelters: open switch, free vs total with inbound, water/fuel/cold chips (clickable), no-medical tag.
HOW IT WORKS	Everything is a direct action: toggling a switch dispatches an action, the planner re-runs, and the cards update from the next snapshot.
DATA / CODE	components/admin/ResourceRail.tsx · lib/actions.ts
CHANGE IDEAS	Rider shifts and availability windows; per-shelter opening hours; show shelter demand history.

Dispatch timeline

SHOWS	A Gantt: one row per in-flight leg (and one per waiting batch) against a 'now' cursor and red **exp** markers where each safe window closes.
HOW IT WORKS	Window = now−3 min to the latest ETA/expiry (clamped 30-150 min ahead). Bars are tinted by survival with a progress fill. Waiting batches show the gap until they would spoil.
DATA / CODE	components/admin/Timeline.tsx
CHANGE IDEAS	Zoom and drag the window; click a bar to focus the leg; show planned vs actual.

Live feed

SHOWS	Time-stamped, colour-coded events: planned, accepted, picked up, delivered, ghost warnings, lease expiries, recoveries, diverts, blocked dispatches, stranded riders.
HOW IT WORKS	Engine-written plain-English messages (last 150 sent to the browser). The same log feeds the live board and the rider's personal activity panel.
DATA / CODE	components/admin/EventFeed.tsx · state.events

Chaos panel

SHOWS	Ghost a rider, close a shelter, citizen fills a shelter, traffic jam, tight-expiry batch, four scenario spawners (banquet, infant formula, cold-chain meds, evening rush), pause/resume, speed 1×-8×, +5 min, reset.
HOW IT WORKS	Each tile dispatches a real action; 'busiest' targets are chosen from live legs. Nothing is animated or faked.
DATA / CODE	components/admin/ChaosPanel.tsx · sim.ts (ghostRider, closeShelter, surgeDemand, trafficJam)
CHANGE IDEAS	Add 'rider declines', 'donor cancels', 'flash flood closes a zone', or scripted scenario timelines.

Optimizer Lab (drawer)

SHOWS	Convergence chart (fitness gain over greedy), and a card per algorithm with expected safe units (+ gain), survival %, travel, solve time, population diversity, and an Apply button.
HOW IT WORKS	Every planning run solves with all four algorithms and stores the results; Apply switches the active algorithm and re-plans. Hybrid is the default.
DATA / CODE	components/admin/OptimizerLab.tsx · engine/solver.ts · Recharts
CHANGE IDEAS	Let judges tune weights live; replay a run step by step; compare on stored historical problems.

Dispatcher Copilot (drawer)	
SHOWS	Chat with suggestions. A command like 'Close Hadapsar' returns a proposed plan card: action chips, before/after KPIs, a list of changes (diverts, releases, new assignments, strandings), and Approve / Dismiss.
HOW IT WORKS	The model (or offline rule parser) returns actions restricted to an allow-list. The server simulates them on a cloned store, diffs the result, and only on approval applies them through the normal action path. Questions ('Why was Aarav chosen?') get an explanation, no actions.
DATA / CODE	components/admin/Copilot.tsx · lib/copilot.ts · api/copilot/route.ts
CHANGE IDEAS	Voice input; multi-step plans; explain-with-evidence linking to the Optimizer Lab; undo.

Guided demo (drawer)	
SHOWS	Seven ordered steps: banquet, ghost, fill shelter, close shelter, infant-formula dependency, copilot, optimizer lab; each with a Run button and a tick driven by real state.
HOW IT WORKS	Ticks are computed from live metrics and events, not from clicks, so they reflect what actually happened.
DATA / CODE	components/admin/DemoGuide.tsx

5.2 Donor portal /donor

Intake	
SHOWS	Organisation picker, free-text box with microphone dictation, example chips, optional photo (drag-drop or pick), 'Analyze with AI'.
HOW IT WORKS	Photos are downscaled to ≤1024 px JPEG in the browser. The server sends text and/or image to the model with a forced tool schema; output is sanitised by zod. Without a key (or on any failure) a rules parser handles text; photo-only returns clearly labelled placeholder values.
DATA / CODE	app/donor/page.tsx · lib/intake.ts · lib/client/image.ts · api/intake/route.ts
CHANGE IDEAS	Barcode/label scanning; multiple items per photo; saved donor templates.

AI draft card	
SHOWS	Editable label, category, quantity, safe minutes, cold-chain checkbox, risk flags, confidence badge (AI vision / Rules parser / Placeholder), rationale and any offline warning.
HOW IT WORKS	Dependency flags are added automatically (formula needs clean water; raw rations need fuel). Posting dispatches immediately.
DATA / CODE	DraftCard in app/donor/page.tsx

Journey of your donation	
SHOWS	Countdown ring, five-step tracker (Posted → Dispatched → Picked up → In transit → Delivered), a mini-map scoped to this batch's donor, riders and shelters, and the riders carrying it with ETA and survival.
HOW IT WORKS	Derived from the batch and its legs; the map uses focus mode (only this batch's entities).
DATA / CODE	Journey in app/donor/page.tsx · LiveMap onlyBatchId
CHANGE IDEAS	Delivery receipts with shelter confirmation; thank-you messages; impact summary per donor.

5.3 Rider portal /rider

Playing as / Take control	
SHOWS	Pick any rider; switch on **Take control** to become human-controlled.
HOW IT WORKS	Manual riders do not auto-accept; the page sends a heartbeat every ~1.8 s. Closing the page or going silent starts the lease countdown.
DATA / CODE	app/rider/page.tsx · setRiderManual, heartbeat actions

Phone frame: offer / active task	
SHOWS	Offer card with 'new delivery offer', quantity, pickup and drop-off, safe-arrival %, accept/decline with a countdown ring; then a three-step progress (Offer → Pickup → Drop-off) and ETA.
HOW IT WORKS	Reads the rider's active leg; Decline releases the leg and re-offers elsewhere (3 min cooldown). A reroute banner appears for 10 sim-minutes after a divert.
DATA / CODE	TaskCard in app/rider/page.tsx
CHANGE IDEAS	Turn-by-turn directions; photo proof at pickup/drop; earnings or impact tally.

Silent / ghost controls	
SHOWS	'Simulate: I'm stuck / unresponsive' button; a rose card with lease time remaining and 'I'm back'.
HOW IT WORKS	Sets the rider silent: heartbeats stop and movement freezes, exercising the whole ghost-recovery path from the rider's own phone.
DATA / CODE	setRiderSilent action

Your activity	
SHOWS	A personal feed of events involving this rider.
HOW IT WORKS	Filters the event log by rider id.
DATA / CODE	app/rider/page.tsx

5.4 Shelter portal /shelter

Capacity gauge and controls	
SHOWS	Radial gauge of free capacity (net of promises to inbound riders), Accepting/Closed switch, Mark full, Reset capacity, a slider for remaining capacity, and '−10/−25/−50' neighbour-drop buttons.
HOW IT WORKS	Each control dispatches an action; the planner immediately diverts inbound riders if their destination became invalid.
DATA / CODE	app/shelter/page.tsx · setShelterOpen, setShelterRemaining, citizenDrop
CHANGE IDEAS	Per-category capacity (beds vs meals); staffed hours; request specific supplies.

What we can safely receive	
SHOWS	Switches for clean water, cooking fuel and cold storage, each labelled with what it unlocks or blocks.
HOW IT WORKS	Facility flags feed the compatibility rules directly (formula ↔ water, raw rations ↔ fuel, cold-chain medicine ↔ cold storage).
DATA / CODE	setShelterFacility action · domain/rules.ts

Inbound deliveries, map, recently received	
SHOWS	Queue with quantity, rider, state, ETA, survival and life-left badges, 'rerouted to you' tag; a map scoped to this shelter; last five received.
HOW IT WORKS	Derived from active and delivered legs for this shelter.
DATA / CODE	app/shelter/page.tsx

5.5 Impact, Live board, Landing and Judge mode

Impact /impact	
SHOWS	Headline stats; safe deliveries over time (delivered vs spoiled); optimizer expected-safe bars with value labels (zero-based axis); shelter capacity used; rider reliability vs delivered; scale cards (riders per 1,000 units/hour computed from observed trips) and the real-world roadmap.
HOW IT WORKS	Charts read metrics.series (one point per sim-minute) and the latest optimizer run. Riders-per-1,000 = (1000 ÷ average units per leg) × (average leg minutes ÷ 60).
DATA / CODE	app/impact/page.tsx · Recharts
CHANGE IDEAS	Add cost-per-meal, CO_2 avoided, donor leaderboards, exportable report.

Live board /live	
SHOWS	Projector-friendly view: cinematic map, large counters, 'Happening now' highlights, waiting-for-dispatch countdowns, average ghost recovery time. No controls.
HOW IT WORKS	Filters the event log to notable events; slow camera sway; larger type.
DATA / CODE	app/live/page.tsx

Landing /	
SHOWS	Hero with a live map backdrop, four role cards, live counters, links to Judge mode and the Live board.
HOW IT WORKS	Uses the same live store, so counters move.
DATA / CODE	app/page.tsx

Judge mode /judge	
SHOWS	The control room plus rider, shelter and donor portals tiled together.
HOW IT WORKS	Iframes on one shared city: act in one pane and watch the others react.
DATA / CODE	app/judge/page.tsx

6 · Data model, API and seed data

Core entities

Entity	Key fields	Notes
Batch	donor, category, qty, unit, coldChain, createdAt, expiresAt, deliveredQty, spoiledQty, unplacedQty, status, recovery[]	Categories: cooked, raw_ration, medical, infant_formula, kit. Status: pending, planned, in_progress, delivered, partial, at_risk, spoiled
Leg	batch, rider, shelter, qty, state, offerExpiresAt, leaseTtl, phase and route endpoints, dropEta, standbyRiderId, survival, divertedFrom	States: offered, to_pickup, to_drop, delivered, released, failed. One active leg per rider
Rider	capacity, speed, reliability, status, lastHeartbeat, silent, manual, cooldownUntil	Status: idle, busy, ghosting, offline
Shelter	capacityRemaining, totalCapacity, accepts[], facilities{water, fuel, cold}, open	'Available' capacity = remaining − promised to active legs
Metrics	delivered, spoiled, ghostRecoveries, diverts, blockedDispatches, response time sum/count, per-minute series	Feeds KPI strip and Impact

API surface

Route	Purpose
GET /api/state	Current snapshot (polling fallback and tests)
GET /api/stream	Server-sent events: a snapshot every tick or action
POST /api/action	Validated actions: reset, pause/resume, setRate, step, setAlgo, replan, addBatch, accept/declineLeg, heartbeat, setRiderManual/Silent, setShelterOpen/Facility/Remaining, citizenDrop, chaos* and scenario
POST /api/intake	Photo/text → structured donation draft (AI with offline fallback)
POST /api/copilot	{message} → reply + proposed actions + simulated diff; {apply} → validated application

Pune seed data (simulated)

Shelters: Kasba Peth Night Shelter (50), Yerwada School Relief Camp (120), Kothrud Community Hall Camp (80, no cooking fuel), Hadapsar Relief Centre (70, no clean water, has cold storage and accepts medical), Warje Flood Relief Hall (60), Kondhwa Relief School (90, cold storage, medical). **Donors:** Koregaon Park banquet hall, Camp hostel mess, Deccan pharmacy, Viman Nagar caterers, Aundh hostel mess. **Riders:** ten volunteers across the city (capacity 25-60, reliability 74-97%). Names and capacities are fictional; coordinates are approximate neighbourhood centres. The seed intentionally includes a shelter without water and one without fuel so dependency rules are demonstrable.

7 · Testing and quality

Tests are designed per phase so every change can be verified quickly.

Suite	Count	What it proves
Domain	11	Geo maths, compatibility rules, seeded RNG, seed data sanity
Greedy and validator	19	180-meal split, shelter capacity, dependency blocking, validator rejections
Simulation scenarios	18	Banquet delivery, ghost recovery, manual accept/decline, diverts, closures, stranded riders, traffic, expiry, conservation on every tick
Optimizers and Monte Carlo	17	Decoder always yields valid plans, never worse than greedy, deterministic, convergence monotone, survival behaviour
Actions API	11	Validation errors, unknown ids, scenarios, pause/rate, algorithm switch, manual rider flow
Intake parser	9	Categories, quantities, times, cold chain, dependency flags, clamping, model-output sanitiser
Copilot	12	Offline understanding, allow-list, preview does not mutate, preview matches approval
Fuzz	8 seeds	Random failures; invariants: no leaks, no overload, no double-booked riders, no unsafe destinations. Extra 60-seed sweeps found and fixed a real bug
Browser (Playwright)	48	Every portal loads without errors; chaos buttons, copilot approval, rider accept, shelter close, donor intake, guided demo; no horizontal overflow at 390 px

Commands: `npm test` · `npm run typecheck` · `npm run lint` · `npm run build` · `npm run e2e` (with the dev server running).

Resilience features

- No AI key: rules-based intake and copilot. No map tiles: plain canvas with live markers and routes. No routing service: straight-line routes.
- SSE failure: automatic polling. Reset button restores the seeded city at any time.
- Mobile-safe layouts verified automatically; light and dark themes.

8 · Limitations and ideas for further changes

Grouped by area so you can pick what to change next.

Known limitations

- State is in memory on one server process; a restart resets the city.
- One trip per rider per plan (no chained multi-pickup routes); diverts move the whole load to a single shelter.
- Distances use haversine × a constant road factor; live traffic is a single city-wide multiplier.
- Optimizer gains are modest on small, unconstrained scenarios.
- The live AI paths (vision intake, copilot tool-use) are written and type-checked but need an API key to exercise; the offline paths are fully tested.
- Public map tiles and routing are third-party services; they were not reachable in the build sandbox, so only the fallbacks were visually verified there.

Ideas by area

Area	Possible changes
Visual design	Apply your design references via tokens; custom marker set; map style tuned to brand; micro-interactions on divert/ghost events; motion for the stranded state

Area	Possible changes
Control room	Cluster and heat layers; draggable timeline; per-batch actions (expedite, cancel, manual assign); saved views; alert sounds for critical events
Chaos and demo	Scripted scenario timelines; 'surprise me' random failure; replay a session; export a demo run as a report
Engine	Multi-pickup routes; partial diverts; time-varying traffic by zone; rider shifts; cold-chain temperature decay; priority classes (medical first); fairness across shelters
Optimizers	Tunable fitness weights in the Lab; larger instances with niching; compare on stored problem sets; adaptive QIGA rotation tables; explain-why for each assignment
AI	Voice intake; multi-item photos; expiry reading from labels; copilot with multi-step plans and undo; anomaly alerts on rider reliability drift
Riders	Real navigation; photo proof; earnings/impact tally; offline-tolerant heartbeats via SMS or WhatsApp fallback
Shelters and donors	Per-category capacity; opening hours; supply requests; donor receipts and impact summaries; recurring donations
Platform	Postgres + queue behind the store; zone sharding; authentication and roles; audit trail export; deployment to a public URL
Analytics	Cost per safe unit, CO_2 avoided, response-time percentiles, per-shelter fill forecasting, exportable PDF report

Where things live

To change…	Start here
Rules (what may go where)	src/lib/domain/rules.ts, src/lib/engine/feasibility.ts
Optimizer behaviour or weights	src/lib/engine/optimizer/*, scorePlan in feasibility.ts, solver.ts
Ghost, lease, divert behaviour	src/lib/engine/sim.ts (releaseLeg, findInvalidLegs, replan, tick)
Scenarios and seed data	src/lib/domain/seed.ts, spawnScenario in sim.ts
Look and feel	src/app/globals.css (tokens), src/components/ui/*
Map behaviour	src/components/map/LiveMap.tsx, src/lib/client/routing.ts
Any panel	src/components/admin/* or the relevant src/app//page.tsx
AI prompts and schemas	src/lib/intake.ts, src/lib/copilot.ts